SDA WEEK 4

WEEK OUTLINE

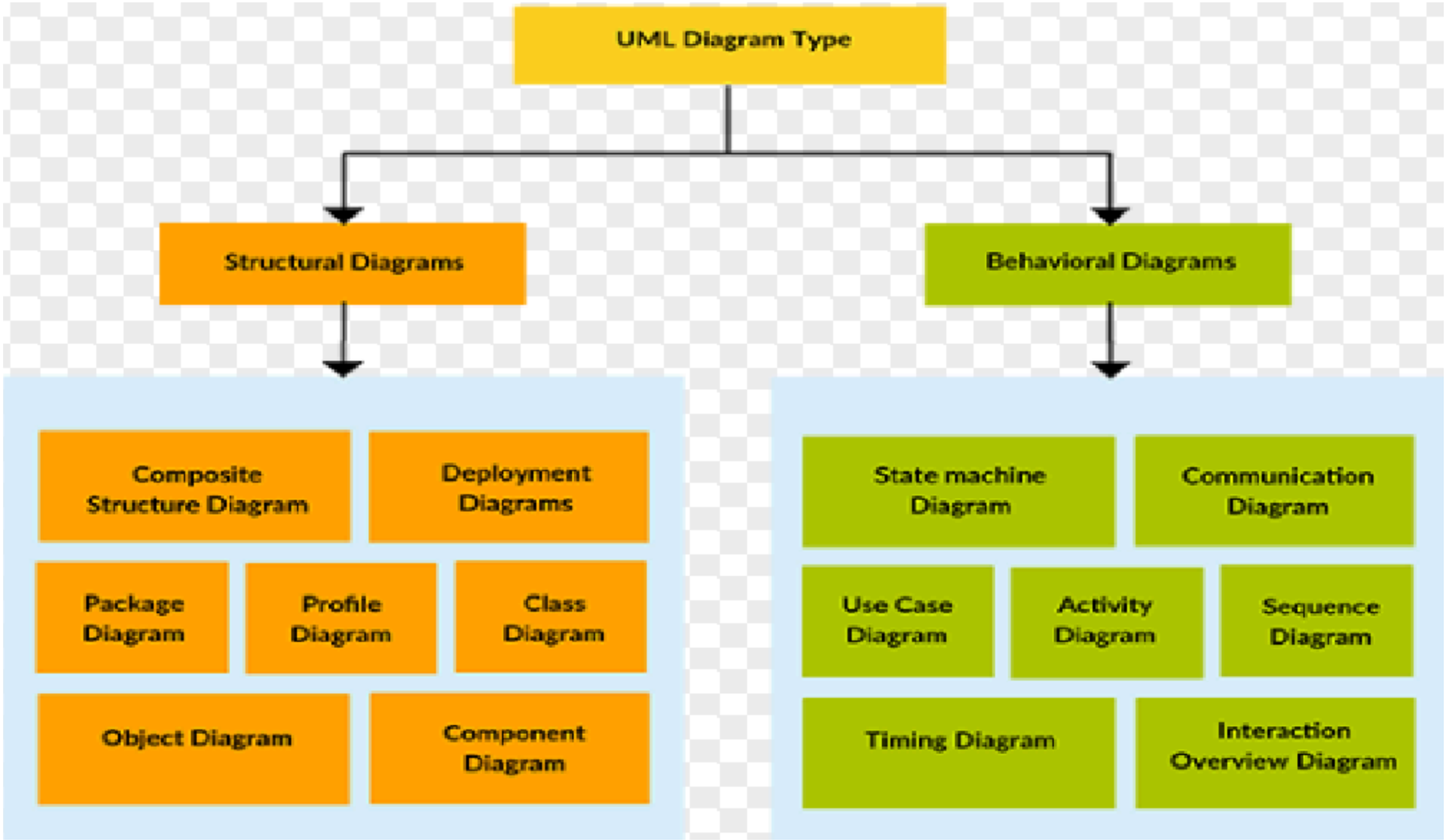
- Class Diagram
- Components of Class Diagram
- Relationships
- Exercises

TYPES OF UML DIAGRAMS

UML diagrams are broadly categorized into two types:

1. **Structure Diagrams:** Like the Class Diagram. They show the 'nouns' of the system–the things that exist, their attributes, and their static relationships. It's about the building blocks and how they are connected, before any action happens.
2. **Behavior Diagrams:** Like the Use Case and Sequence Diagrams we'll see later. They show the 'verbs'–the functionality, the interactions, and the flow of control over time.

Types of UML Diagrams



WHAT IS UML CLASS DIAGRAM?

So, what exactly is a class diagram? The best way to understand it is with an analogy: a **family tree**. Imagine I ask you to draw your family tree. What would you do?

- You'd identify the main members: grandparents, parents, siblings. (These are our classes)
- You'd figure out how they are related: who is married to whom, who is whose child. (These are the relationships)

WHAT IS UML CLASS DIAGRAM? ...

- You'd list their characteristics: name, age, birthdate. (These are the attributes)
- You'd think about what they do: their job, their role in the family. (These are the operations)
- You might even see how traits are inherited. (This is generalization)

A class diagram does exactly this, but for a software system. It's the family tree of your code.

ELEMENTS OF A CLASS DIAGRAM

A class diagram is composed primarily of the following elements that represent the system's business entities:

- **Class**
- **Class Relationships**

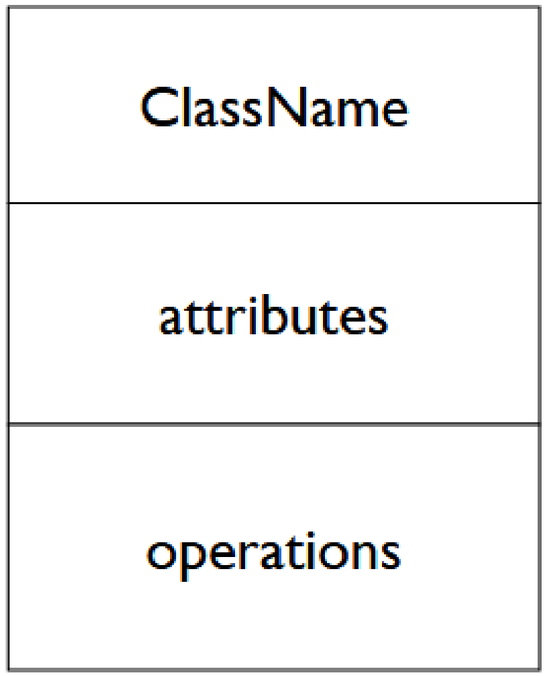
DECONSTRUCTING A CLASS ...

A class is a blueprint or a template for creating objects. It encapsulates together both the data (attributes) that an object will hold and the behavior (operations) that it can perform.

It is drawn as a rectangle, almost always divided into three compartments.

DECONSTRUCTING A CLASS ...

- The Name Compartment
- The Attributes Compartment
- The Operations Compartmen



DECONSTRUCTING A CLASS ...

The Name Compartment: The top compartment is mandatory. It simply holds the name of the class. It should be a singular noun (e.g., Student, Account, Sensor) and should clearly represent the concept it's modeling. By convention, we use PascalCase (capitalizing the first letter of each word).

DECONSTRUCTING A CLASS ...

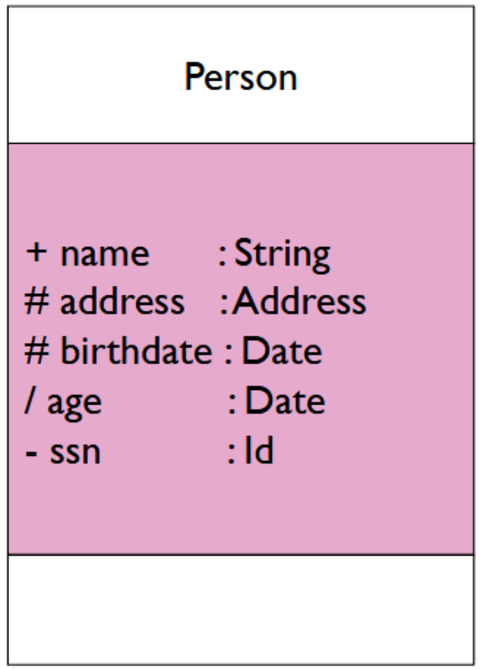
The Attributes compartment: The middle compartment lists the attributes. An attribute is a named property of the class that describes a piece of information that objects of this class will hold.

- For a Person class: name, address, birthdate.
- For a Car class: make, model, year.

The syntax is: attributeName : Type

DECONSTRUCTING A CLASS ...

ATTRIBUTES-VISIBILITY(ACCESS SPECIFIERS):



Attributes can be:

+ public
# protected
- private
~ package
/ derived

DECONSTRUCTING A CLASS ...

ATTRIBUTES-VISIBILITY(ACCESS SPECIFIERS):

- **+ : Public.** Accessible by any other class.
- **- : Private.** Accessible only within this class. This is the most common. We protect our data.
- **#: Protected:** Accessible within this class and its subclasses (we' ll see this next lecture).
- **~ : Package:** Accessible to other classes in the same package.
- **/: (slash):** Derived. The value of this attribute can be calculated from other attributes (e.g., age can be derived from birthdate and the current date).

DECONSTRUCTING A CLASS ...

The Operations: The bottom compartment lists the operations. These are the behaviors, the actions that objects of this class can perform. In code, these become methods.

- For a Person class: eat(), sleep(), work().
- For a BankAccount class: deposit(amount), withdraw(amount), getBalance().

DECONSTRUCTING A CLASS ...

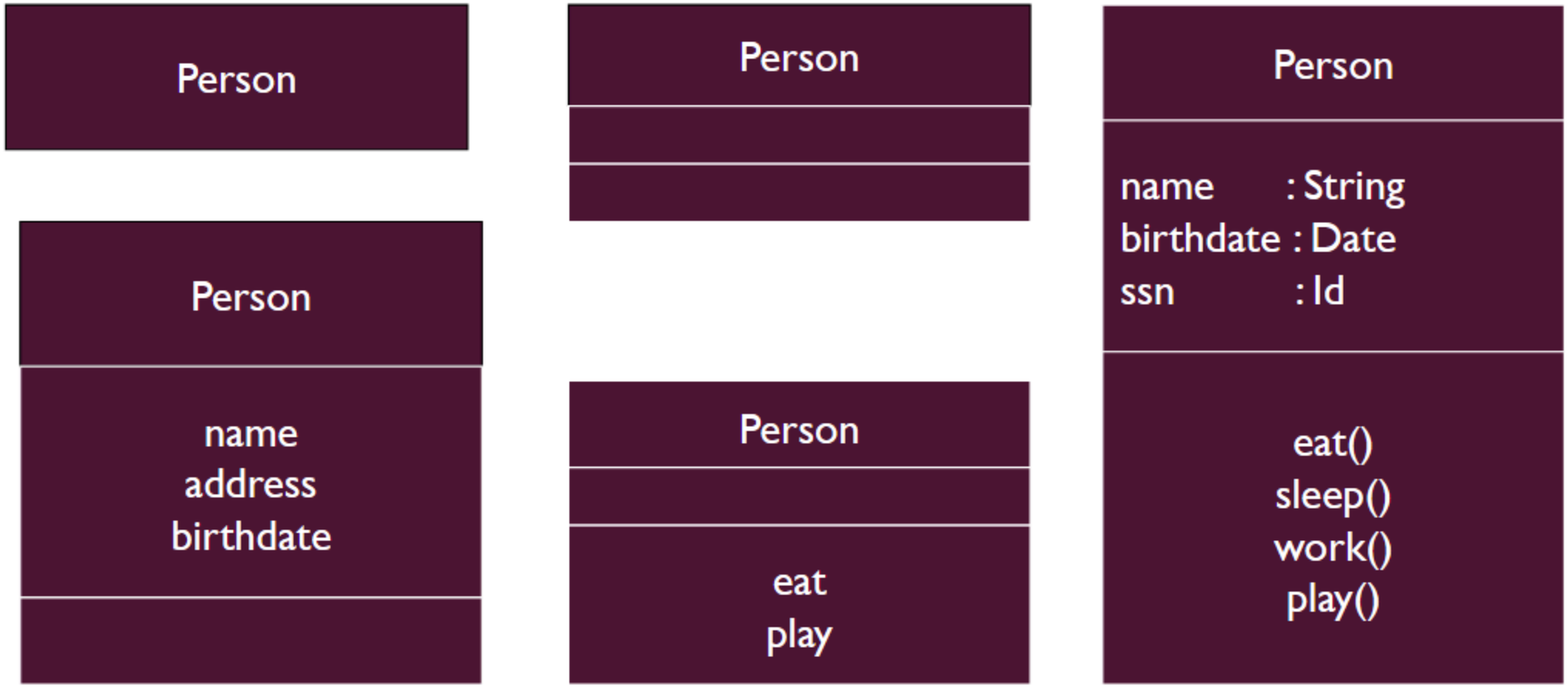
The operation signature/prototype. includes:

- Visibility (+, -, #)
- Name
- Parameters list (in parentheses)
- Return type (after the colon)

For example:

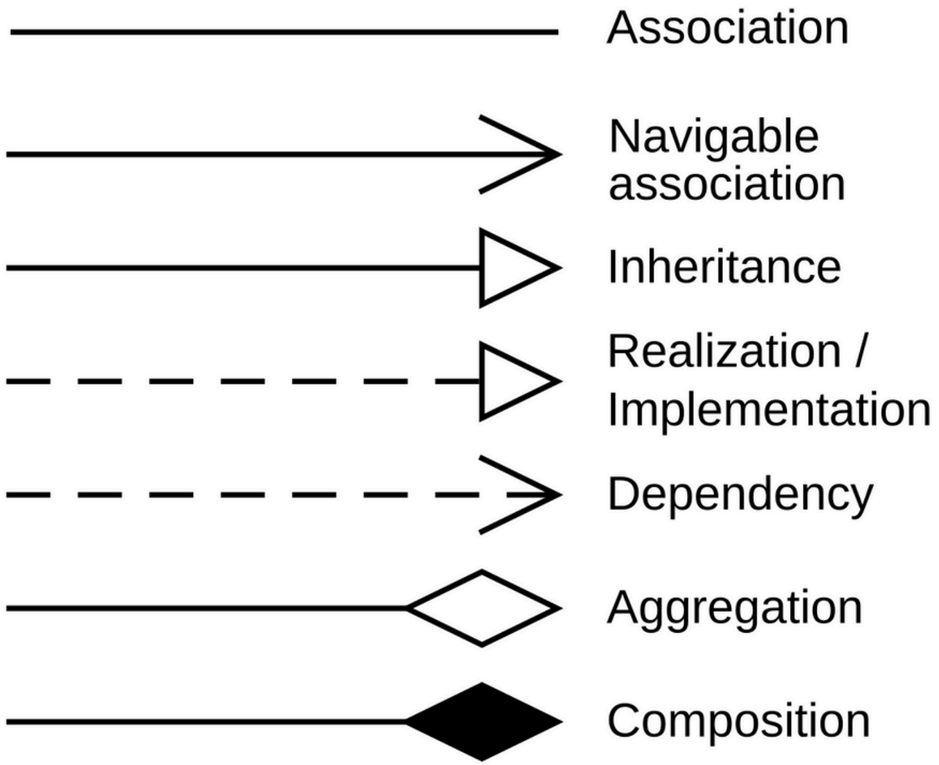
- + newEntry(name : string, age : int) : void
- + getPhone(name : string) : PhoneNumber

When drawing a class, you needn't show attributes and operation in every diagram.



CLASS RELATIONSHIPS

UML defines several key types of relationships. Today, we will cover the first three fundamental ones: Dependency, Generalization, and Association. Understanding the difference between these is critical to good object-oriented design.



DEPENDENCY

- A dependency is a using relationship. It signifies that a change in one class (the supplier) may affect the behavior of another class that uses it (the client). It is a weak, short-term, and often temporary link.
- The most common way a dependency occurs is when a reference to one class is passed into a method of another class. The client class doesn't own the supplier class; it doesn't store it as a permanent attribute. It just uses it briefly for a single task

DEPENDENCY

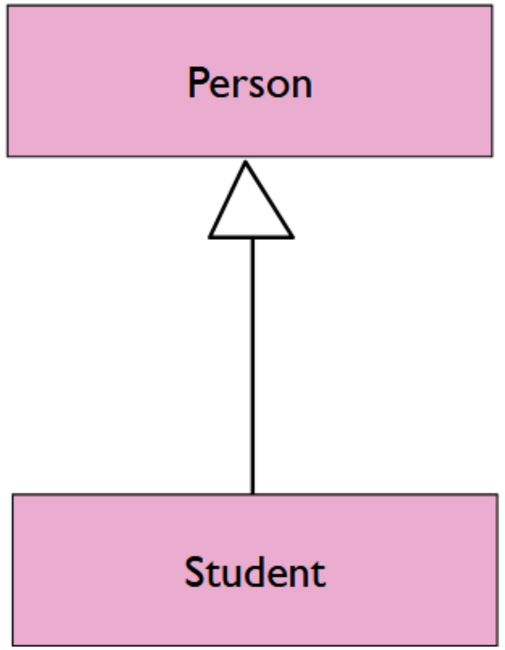
```
class CourseSchedule {
   ...
public:
   void add(c: Course)
   void remove(c: Course)

};
```

```
class Course{
   ...
};
```

CLASS RELATIONSHIPS ...

GENERALIZATION

This is an "is-a" relationship. It connects a subclass to its superclass. The subclass specializes the more general superclass. The key idea is that the subclass inherits all the attributes and operations of the superclass.



GENERALIZATION

A Driver is a Person. Therefore, we can assume that every Driver has a name and an age because every Person does. The Driver class might then add its own specific attributes, like licenseNumber.

```
class Person {
 ...
};
```

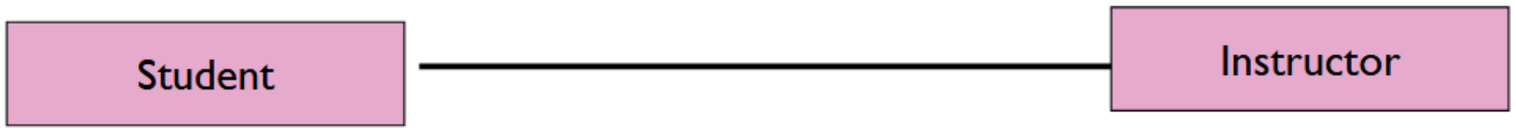
```
class Driver: public Person {
 ...
};
```

```
class Student : public Person {
 ...
};
```

CLASS RELATIONSHIPS ...

ASSOCIATION

An association represents a structural, long-term, "has-a" link between objects of two classes. It implies that the classes know about each other, can communicate, and have a lasting relationship. Unlike dependency, this is not temporary.



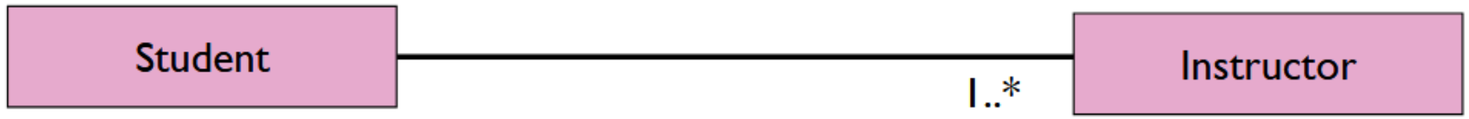
MULTIPLICITY

Multiplicity shows, **How many**? How many Instructor objects does a Student have? How many Student objects does an Instructor have?

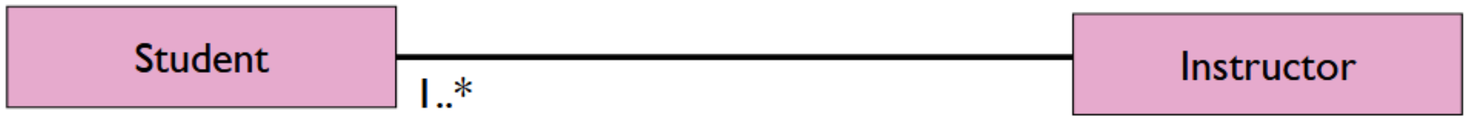
- `1` - Exactly one
- `*` - Zero or more (many)
- `1..*` - One or more
- `0..1` - Zero or one (optional)
- `2..4` - A specific range (between 2 and 4, inclusive)

ASSOCIATION

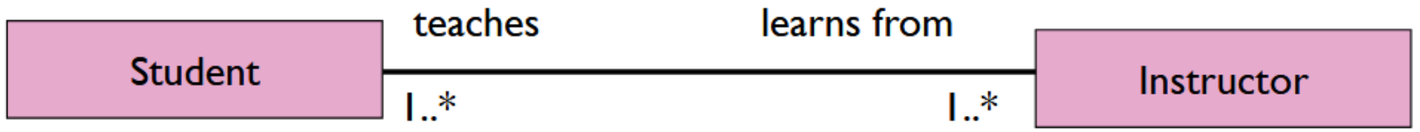
The example indicates that a *Student* has one or more *Instructors*:



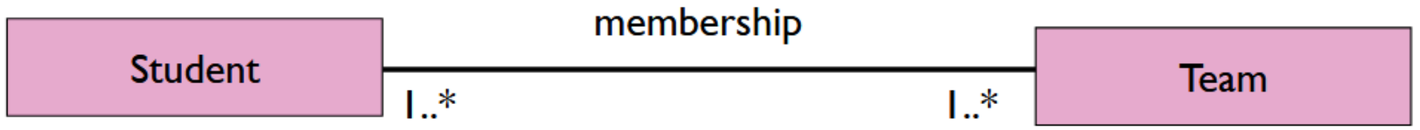
The example indicates that every *Instructor* has one or more *Students*:



We can make associations even more descriptive by using role names and association names



We can also name the association.



KINDS OF ASSOCIATIONS

- Simple Associations
- Aggregation
- Composition

KINDS OF SIMPLE ASSOCIATION

- w.r.t navigation
  - One-way Association
  - Two-way Association
  - Self association
- w.r.t number of objects
  - Binary Association
  - Ternary Association
  - N-ary Association

- With respect to navigation: How can we traverse the relationship? Is it one-way or two-way?
- With respect to the number of objects: How many classes are involved in a single association? Is it between two, three, or more?

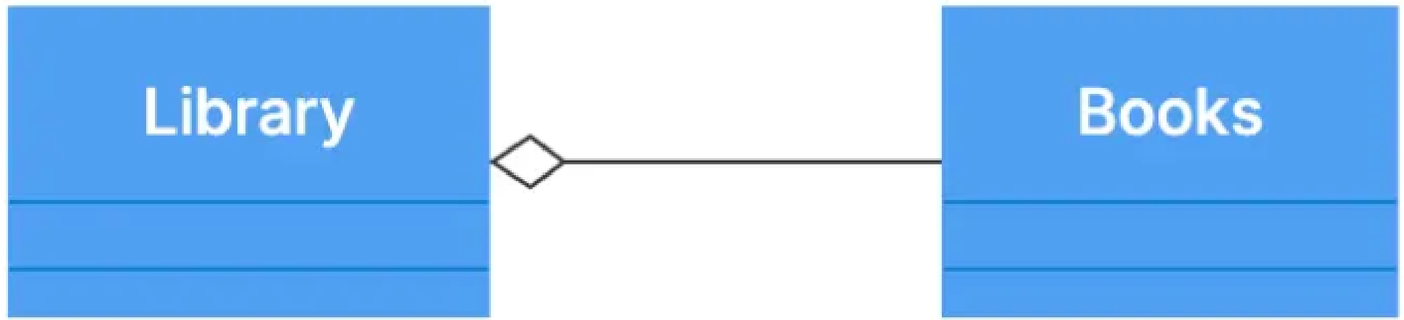
AGGREGATION

The two of the most important specialized forms of association are Aggregation and Composition. Both represent a whole-part relationship.

- **Aggregation:** This represents a relationship where the part can exist independently of the whole. It's a 'loose' ownership.
- Think of a Car and its Engine. The car has an engine. But if you scrap the car, the engine can be removed and placed in another car. The engine's life is not tied to the life of that specific car.

The notation is a hollow diamond on the side of the whole.

AGGREGATION



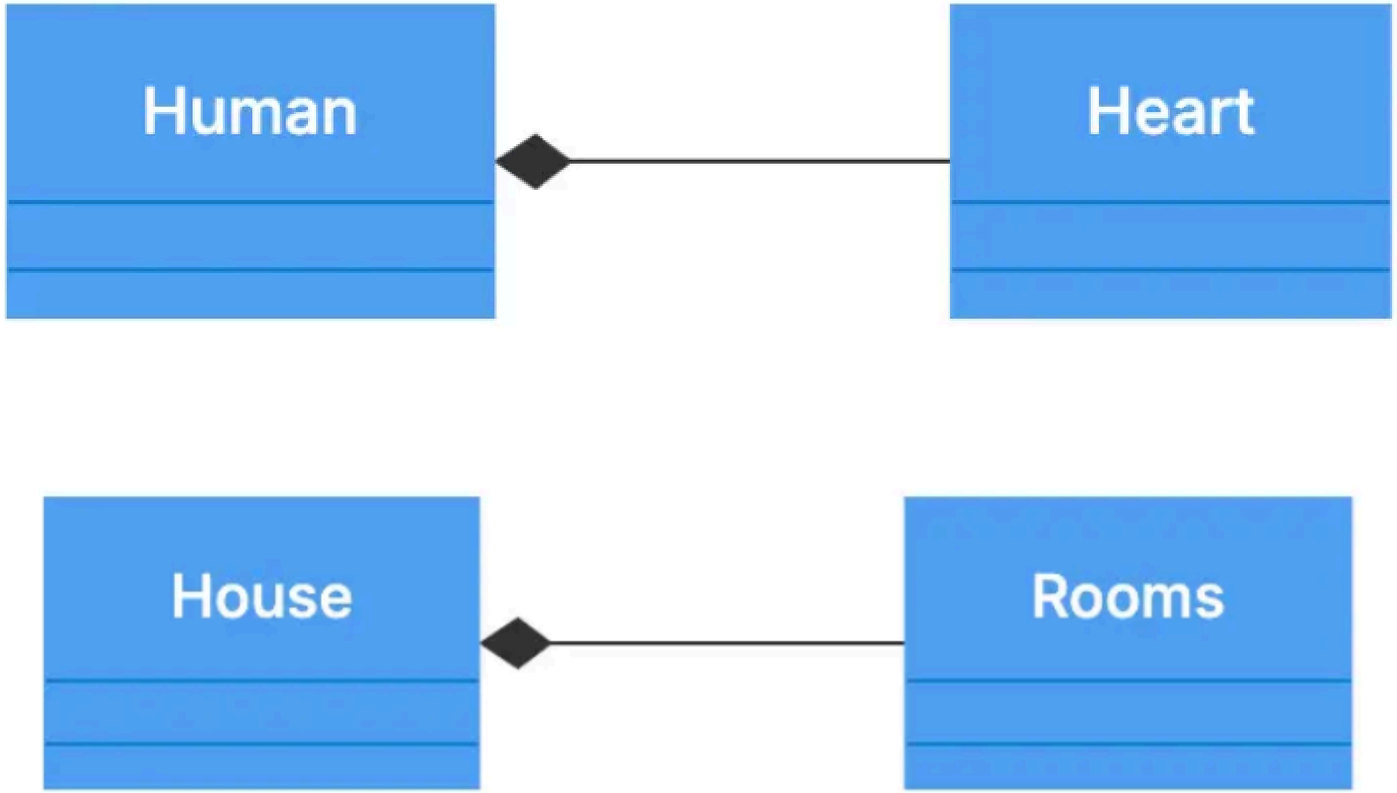
The notation is a hollow diamond on the side of the whole.

COMPOSITION

Now, Composition. This is a much stronger form of ownership. The part cannot exist independently of the whole. The lifetime of the part is controlled by the whole. If the whole is destroyed, the parts are destroyed with it.

The notation is a filled diamond on the side of the whole.

COMPOSITION



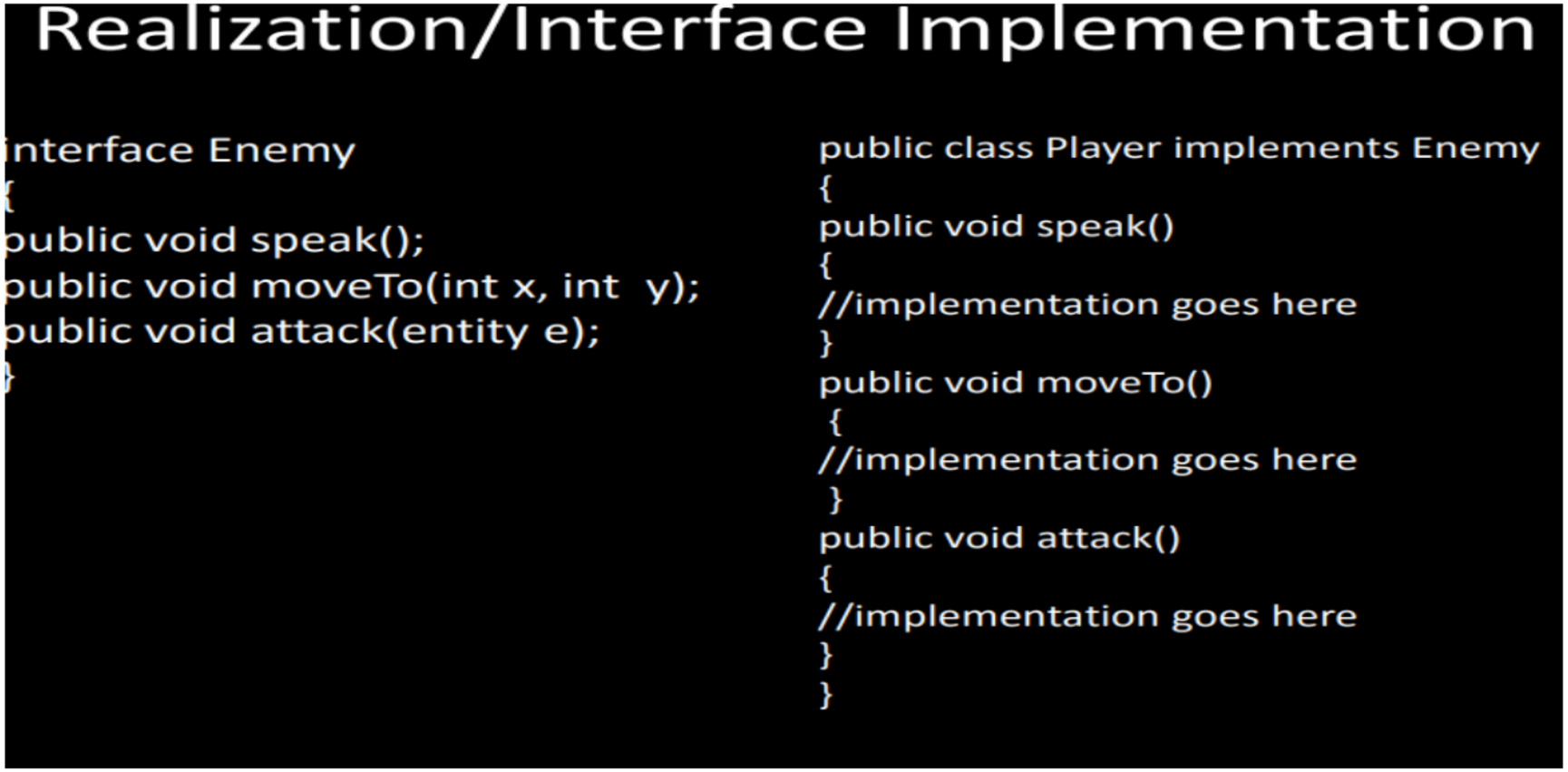
The notation is a filled diamond on the side of the whole.

INTERFACE REALIZATION

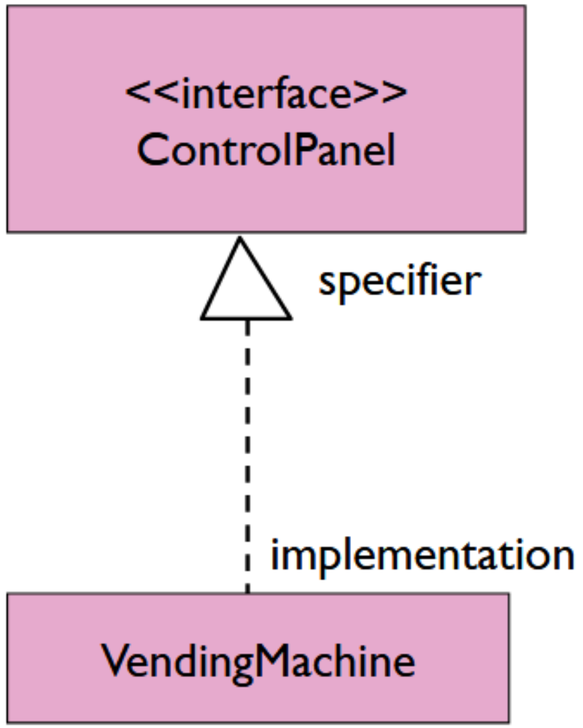
This is the relationship between a class and an interface. The class realizes or implements the contract (the operations) specified by the interface.

The notation is a dashed line with a hollow triangle pointing to the interface. The interface itself is often labeled with «interface».

INTERFACE REALIZATION

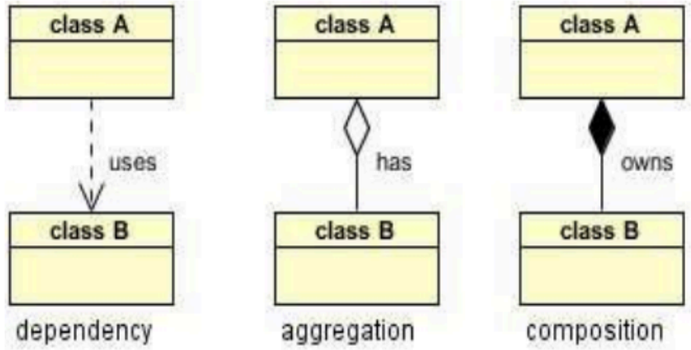


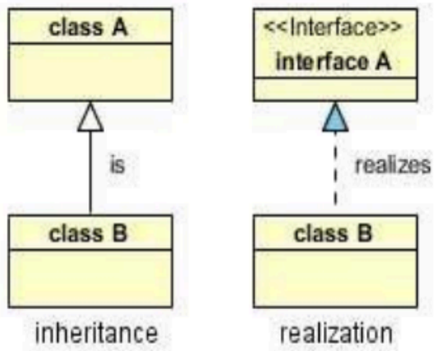
INTERFACE REALIZATION



RELATIONSHIPS IN A NUTSHELL

- Dependency : class A uses class B
- Aggregation : class A has a class B
- Composition : class A owns a class B
- Inheritance : class B is a Class A (or class A is extended by class B)
- Realization : class B realizes Class A (or class A is realized by class B)

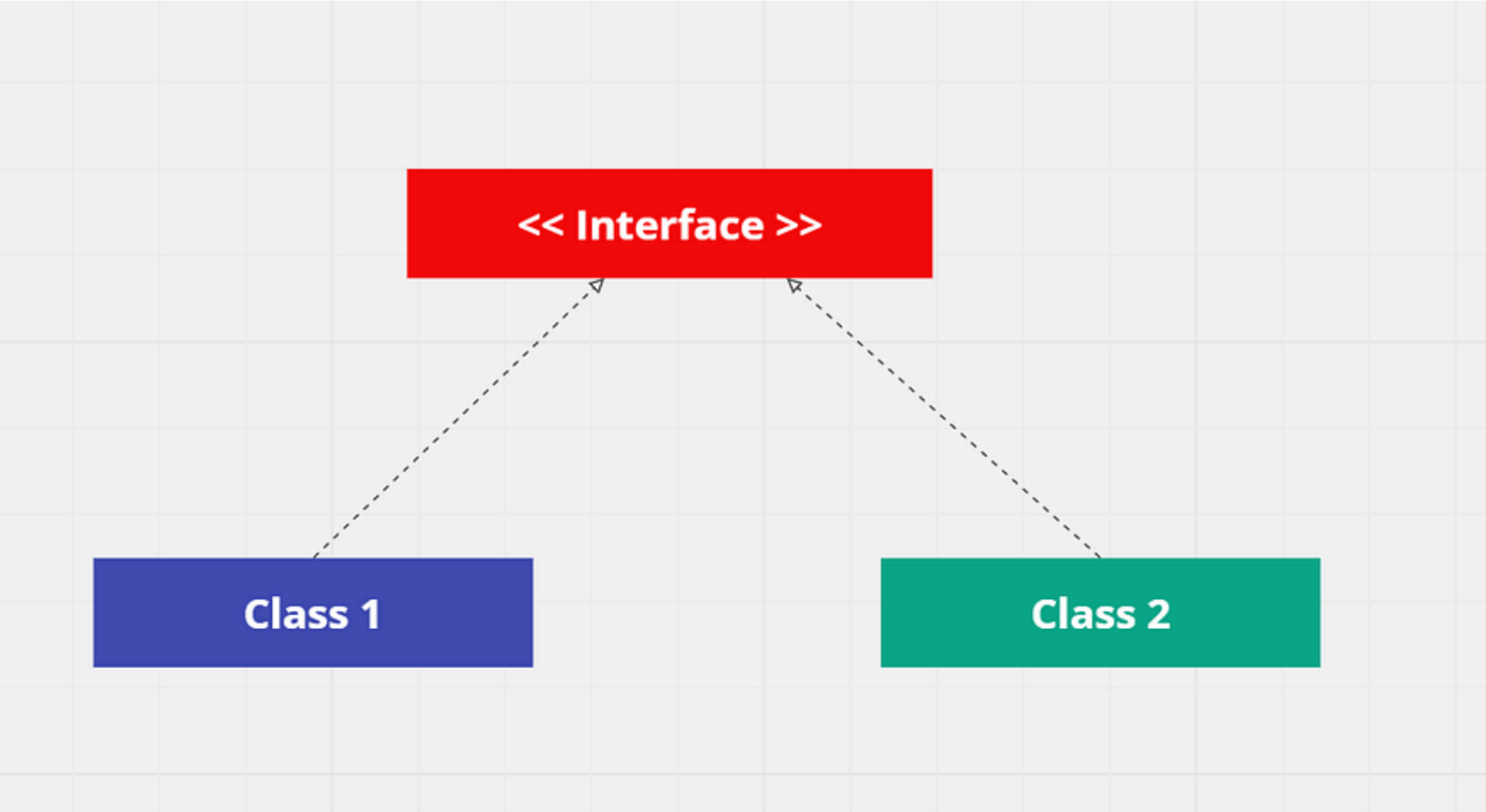




INTERFACE REALIZATION

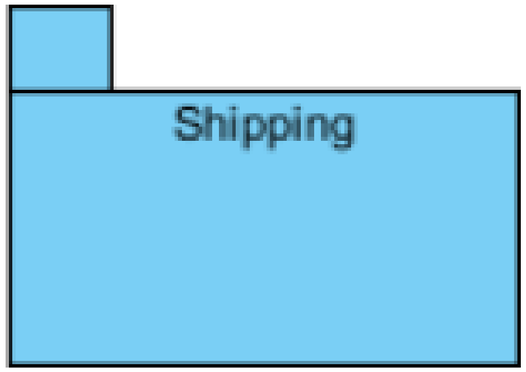
- A parameterized class or template defines a family of potential elements.
- To use it, the parameter must be bound.
- A template is rendered by a small dashed rectan le superimposed on the upper-right corner of the class rectangle. The dashed rectangle contains a list of formal parameters for the class.
- Example: Interface SiteSearch is realized (implemented) by SearchService.

INTERFACE REALIZATION

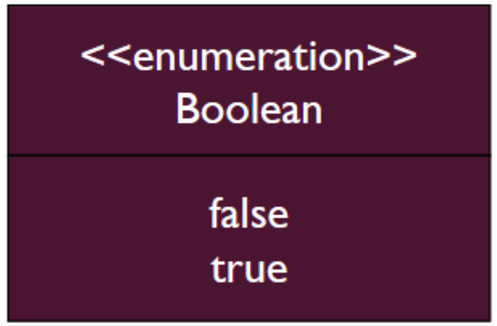


PACKAGE

A package is a container-like element for organizing other elements into groups. A package can contain classes and other packages. Packages can be used to provide controlled access between classes in different packages.

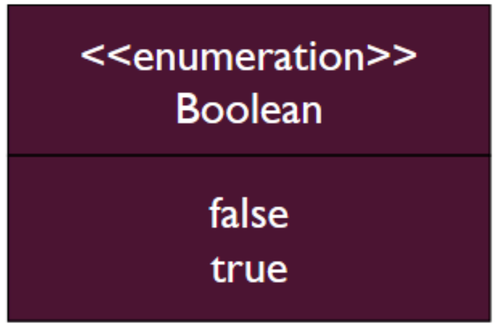


ENUMIRATION



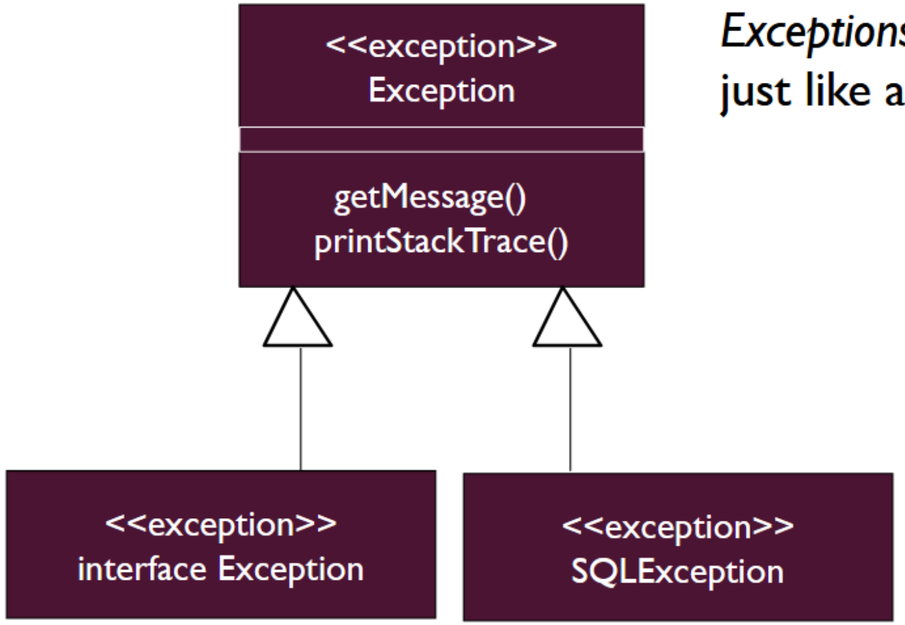
An *enumeration* is a user-defined data type that consists of a name and an ordered list of enumeration literals.

ENUMIRATION



An *enumeration* is a user-defined data type that consists of a name and an ordered list of enumeration literals.

EXCEPTIONS

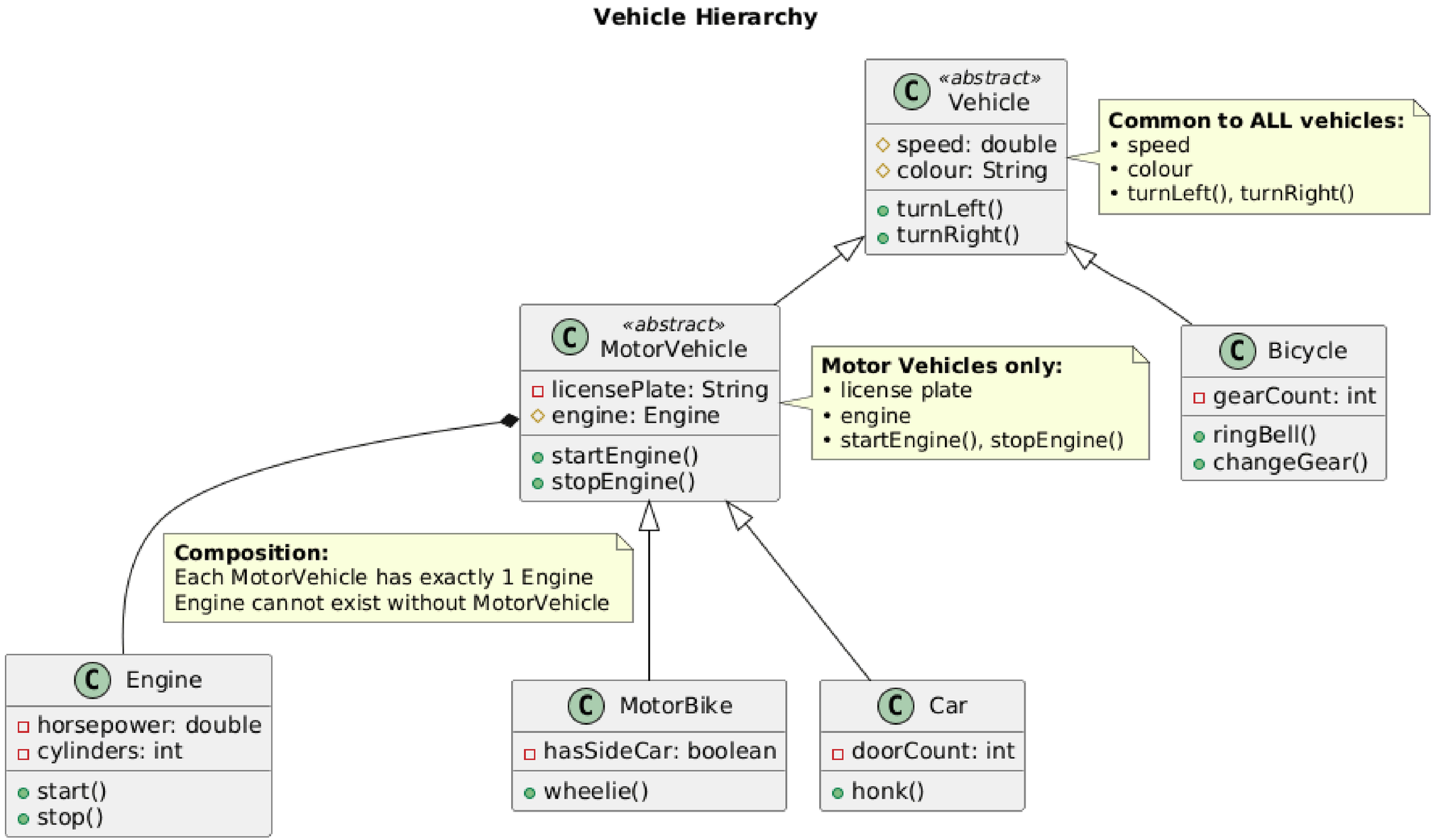


Exceptions can be modeled just like any other class.

EXERCISE

We need to develop an application that models different kinds of vehicles including bicycles, motorbikes, and cars, where all vehicles share common attributes like speed and colour and common behaviors such as turning left and right, with bicycles and motor vehicles both being specific types of vehicles that inherit these common features, while motor vehicles additionally have engines and license plates and are further specialized into motorbikes and cars, creating a clear inheritance hierarchy with a base Vehicle class and distinct subclasses for each vehicle type.

EXERCISE

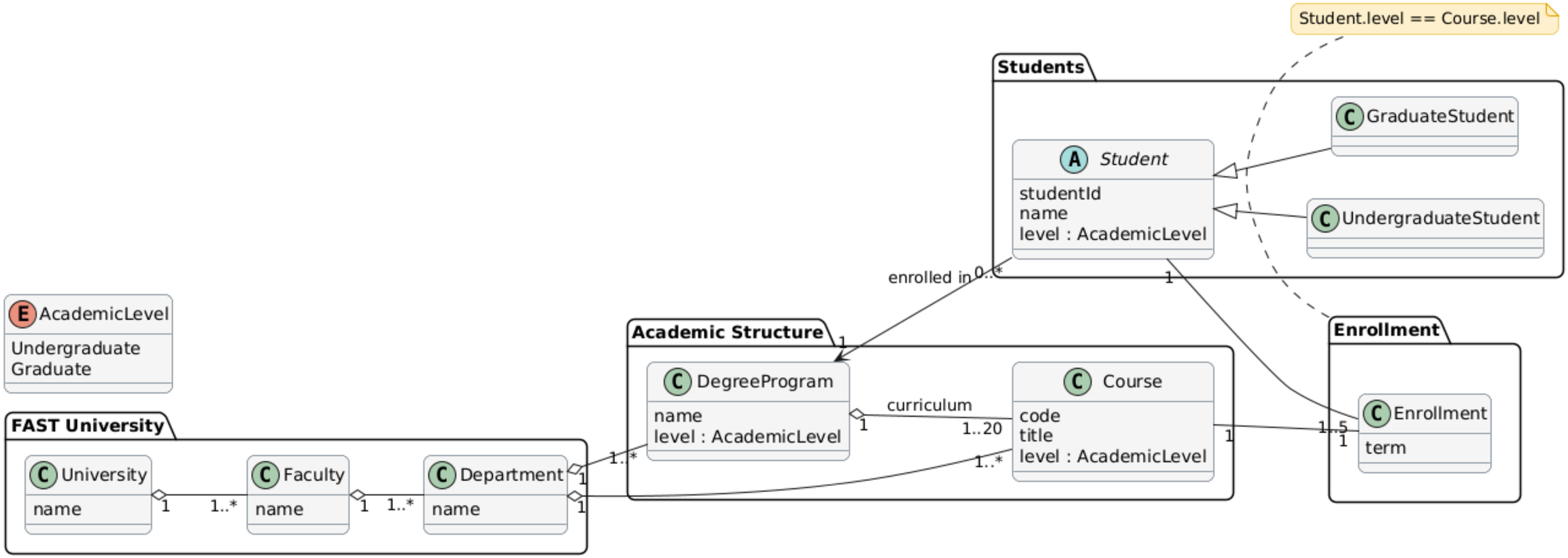

Vehicle Hierarchy
«abstract»
Vehicle
◇ speed: double
◇ colour: String
turnLeft()
turnRight()
Common to ALL vehicles:
• speed
• colour
• turnLeft(), turnRight()
«abstract»
MotorVehicle
licensePlate: String
◇ engine: Engine
startEngine()
stopEngine()
Motor Vehicles only:
• license plate
• engine
• startEngine(), stopEngine()
Bicycle
gearCount: int
ringBell()
changeGear()
Composition:
Each MotorVehicle has exactly 1 Engine
Engine cannot exist without MotorVehicle
Engine
horsepower: double
cylinders: int
start()
stop()
MotorBike
hasSideCar: boolean
wheelie()
Car
doorCount: int
honk()

EXERCISE

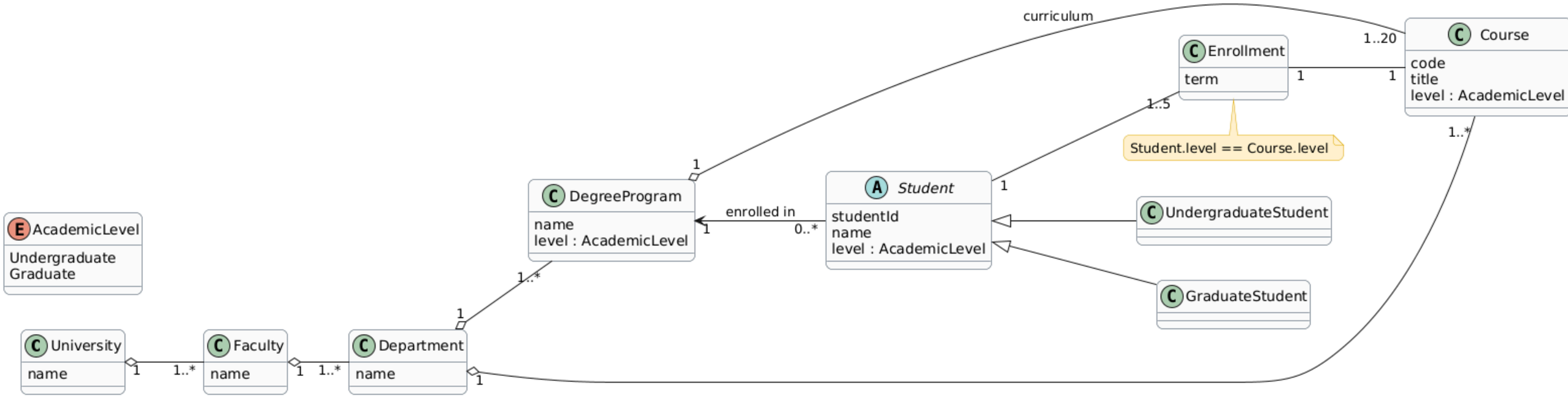
University System:

- FAST university offers degrees to students.
- The university consists of faculties each of which consists of one or more departments.
- Each degree is administered by a single department.
- Each student is studying towards a single degree.
- Each degree requires one to 20 courses.
- A student enrolls in 1-5 courses (per term).
- A course cab be either graduate or undergraduate, but not both.
- Likewise, students are graduates or undergraduates but not both.

EXERCISE SOLUTION



EXERCISE SOLUTION (WITHOUT PACKAGES)



EXERCISE

- Draw the UML class diagram showing the domain model for online shopping. The purpose of the diagram is to introduce some common terms, "dictionary" for online shopping - Customer, Web User, Account, Shopping Cart, Product, Order, Payment, etc. and relationships between. It could be used as a common ground between business analysts and software developers.
- Each customer has unique id and is linked to exactly one **account**. Account owns shopping cart and orders. Customer could register as a web user to be able to buy items online. Customer is not required to be a web user because purchases could also be made by phone or by ordering from catalogues. Web user has login name which also serves as unique id. Web user could be in several states - new, active, temporary blocked, or banned, and be linked to a **shopping cart**. Shopping cart belongs to account.
- Account owns customer orders. Customer may have no orders. Customer orders are sorted and unique. Each order could refer to several **payments**, possibly none. Every payment has unique id and is related to exactly one account.
- Each order has current order status (new, hold, shipped, delivered, closed). Both order and shopping cart have **line items** linked to a specific product. Each line item is related to exactly one product. A product could be associated to many line items or no item at all.

THANK YOU